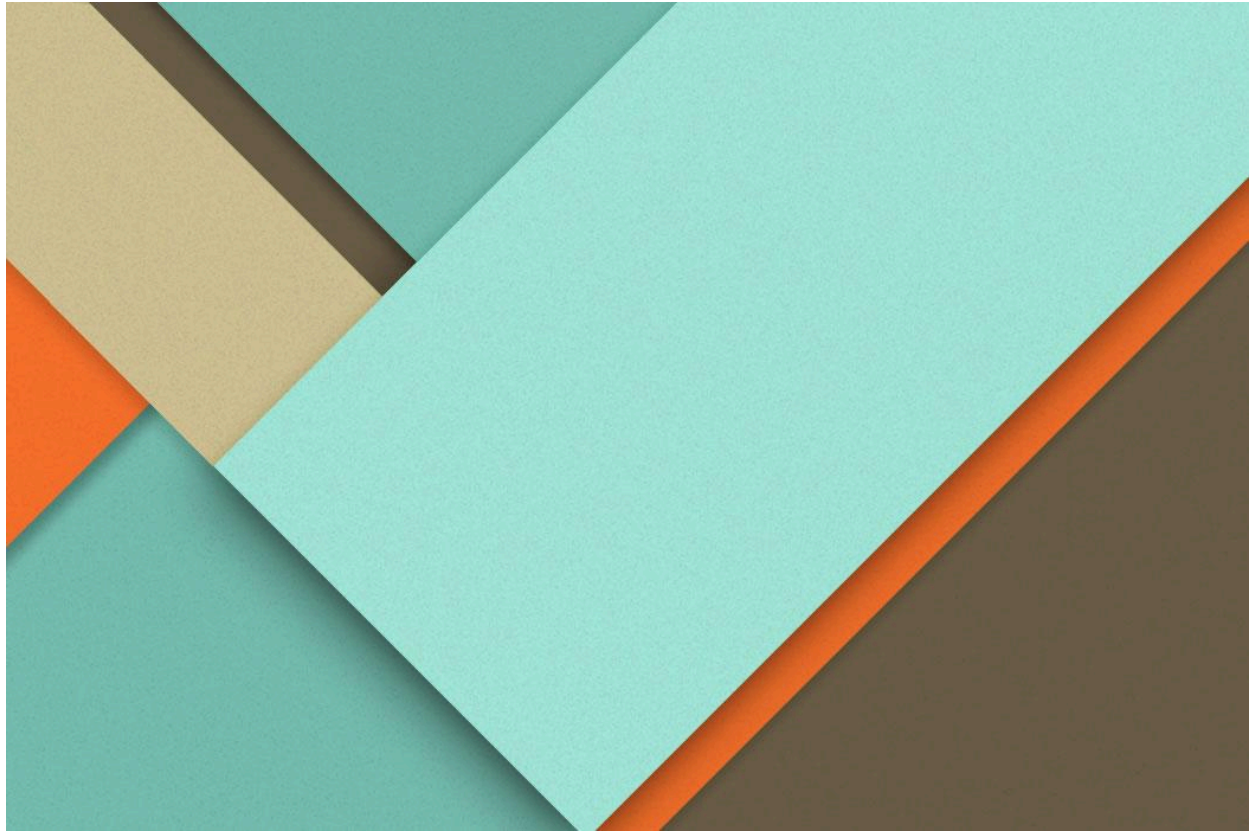




Data Science Portfolio

Hafza Abdullahi

Submission Date: 27/03/2026

—

Project Overview & Technology Reasoning

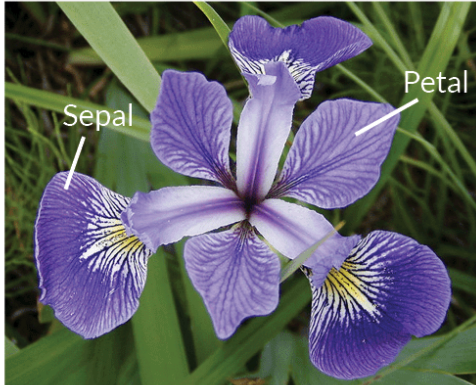
Objective: To develop a professional machine learning portfolio demonstrating proficiency across classical ML, Deep Learning, and Reinforcement Learning.

Technology choices (To be Updated)

1. Python
2. Jupyter Notebooks
3. Scikit-Learn, Seaborn

Dataset One - Iris Flower Dataset

For **chapters 2, 3, and 4** I focused on Classical Machine Learning comparison using the **Iris Dataset**, a dataset containing 3 classes: Setosa, Versicolor, and Virginica. The goal was to observe how different mathematical approaches handle the same data.



Iris Versicolor



Iris Setosa



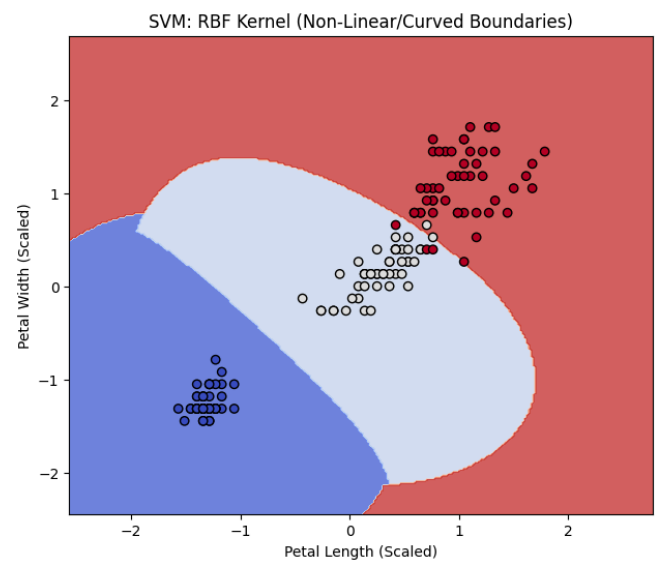
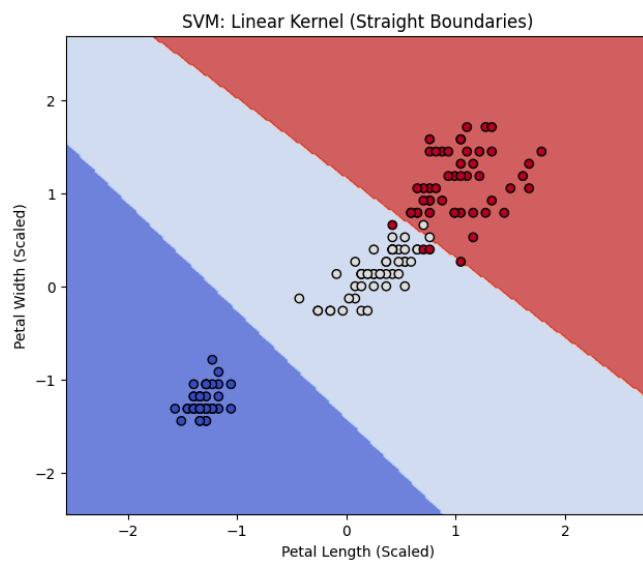
Iris Virginica

Data Processing

- **Adjustment:** Split the dataset into a 70/30 train-test split using a fixed random state (42) for reproducibility.
- **Adjustment:** Applied `StandardScaler` to normalize the features
- **Practical Impact:** Distance-based algorithms (like KNN and K-Means) and margin-based algorithms (like SVM) perform significantly better and converge faster when features share the same scale

Algorithm 1: Support Vector Machines (SVM)

- **Implementation:** Developed two models to compare boundaries: a Linear SVM (this data can be separated by a straight line/plane) and an RBF SVM (using the 'Kernel Trick' for higher dimensions).
- **Analysis & Impact:** After scaling, the Linear SVM achieved an accuracy of 0.98, while the RBF SVM achieved a perfect accuracy of 1.00.
- **Visualization:** I created contour plots isolating Petal Length and Petal Width to visually contrast the straight decision boundaries of the linear kernel against the curved, non-linear boundaries of the RBF kernel.

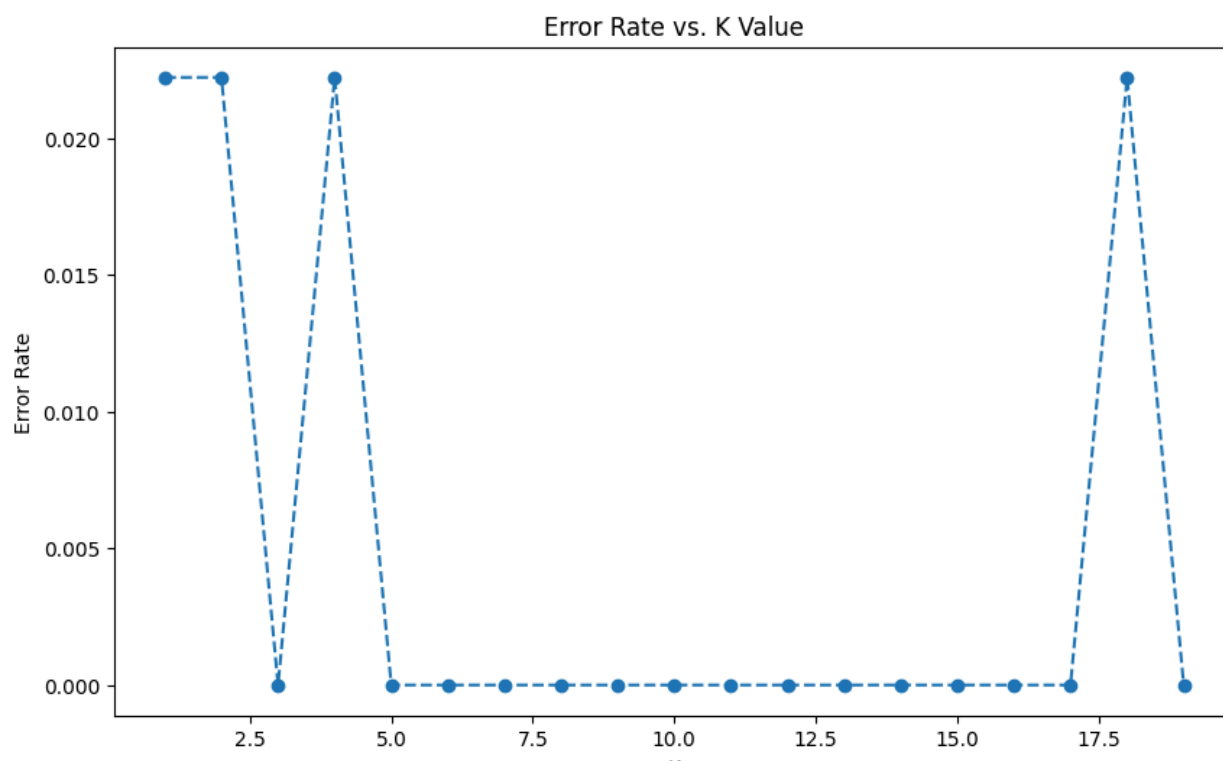


Algorithm 2: K-Nearest Neighbors (KNN)

The primary challenge with KNN is selecting the optimal K value. Plotting the accuracy and error rates against the K values to find the best k value for this dataset

Implementation: Built a KNN classifier and iterated through K values from 1 to 20.

Analysis & Impact: Plotted the accuracy and the Error Rate vs. K Value to systematically identify the optimal number of neighbors without underfitting or overfitting the model.



This graph proves that the KNN model is highly effective for this dataset. The transition from a 2.2% error at K=3 and back to a 0% error at K=5 shows how KNN is susceptible to local noise. A lower K value is better since overfitting to a single dataset is not ideal.

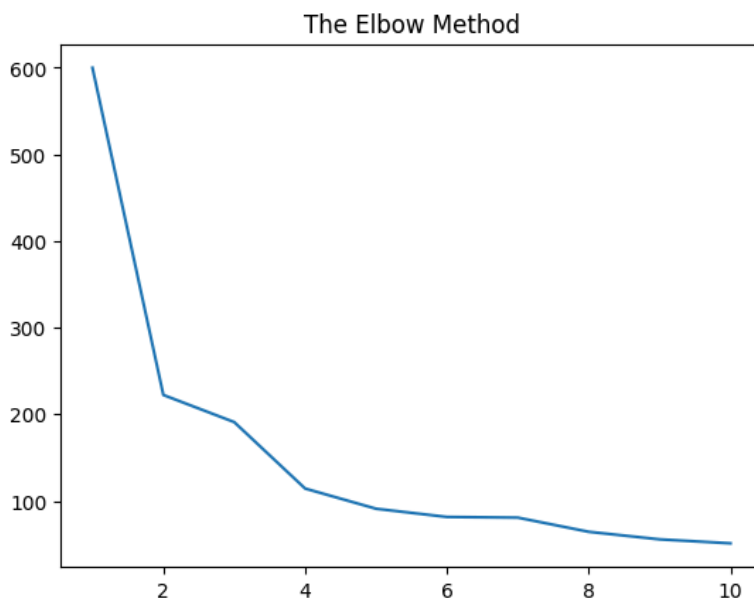
Algorithm 3: K-Means Clustering

Implementation:

Unlike SVM and KNN, which are Supervised Learning meaning the model is told the answers, I implemented K-Means as an Unsupervised Learning algorithm.

- I stripped away the I target species labels and gave the algorithm only the raw measurements (Petal/Sepal length and width).
- I ran the KMeans algorithm for K values ranging from 1 to 10. For each iteration, the model calculated the WCSS (Within-Cluster Sum of Squares), which measures the closeness of the clusters by calculating the distance between points and their respective centroids.

The primary goal of including K-Means is to demonstrate **Pattern Recognition** capabilities without human intervention such as labeled data the algorithm learns from



The Elbow Method

In this graph, a sharp decline can be observed which then flattens out

- Between 1 and 2: There is a massive drop in error. This means 1 cluster was a very **poor fit**.
- Between 2 and 3: There is another significant drop.
- **At k=3:** The bend or **Elbow** occurs and then flattens out. This shows the algorithm is naturally organizing the data into **three distinct groups**.

Plan for Final Portfolio Submission (Due 17th April 2026)

Deep Learning Implementation (Neural Networks)

- **Dataset:** I plan to use the **MNIST Digit Dataset** or the **CIFAR-10 Dataset**. These are significantly more complex than the Iris dataset and require deep learning to achieve high accuracy.
- **Implementation:** I will build a **Multi-Layer Perceptron (MLP)** and a **Convolutional Neural Network (CNN)** using **TensorFlow/Keras**.
- **Analysis:** I will document the impact of "Hyperparameter Tuning" for deep learning, specifically focusing on the number of hidden layers, neurons, and the use of **Dropout layers** to prevent overfitting

Reinforcement Learning (RL)

- **Dataset/Environment:** I will utilize the **OpenAI Gym** library, specifically the **CartPole-v1** or **MountainCar-v0** environment.
- **Implementation:** I will implement a **Q-Learning** or **Deep Q-Network (DQN)** agent.
- **Analysis:** I will maintain a log showing how the agent's "Total Reward" increases over time as it learns from its environment through trial and error, demonstrating the fundamental RL loop of Observation -> Action -> Reward.

Advanced Evaluation & Professional Standards

- **Metrics:** For all models (Classical and Deep Learning), I will move beyond simple accuracy scores. I will implement **Confusion Matrices**, **ROC Curves**, and **Classification Reports** (Precision, Recall, and F1-Score).
- **Code Quality:** I will refactor all notebooks to include modular functions, comprehensive docstrings, and a consistent naming convention as per the assessment's "Professional Standard" requirement.
- **Comparison:** A final summary chapter will be added to compare the performance of Classical ML vs. Deep Learning on identical tasks where applicable.